

Goal:

- Transition from console-based OS emulation to desktop-style GUI using IMGUI (Dear ImGui library)
- Understand immediate-mode GUI paradigm and its advantages for real-time systems
- Implement polling mechanisms for keyboard and mouse input in a desktop environment
- Design responsive UI components that mirror modern OS interfaces

Features to design in our CSOPESY emulator

- A means to support keyboard input, both real-time and event-driven.
 - Real-time = The user can always type characters, and this displays on-screen immediately. E.g. Word processor programs.
 - This is also known as immediate-mode UI.
 - Event-driven = wait for an event, such as an enter command, and then perform an associated operation.
- How an application must behave under real-time/event-driven conditions.
 - Real-time = screen always refreshes even if there’s no event/user input.
 - Sample applications: games and interactive applications like streaming websites.
 - Sample OSes: Modern OS like Windows are considered real-time.
 - Event-driven = screen refreshes after user input.
 - Sample applications:
 - Sample OSes: console/command line interfaces. However, there are means to support real-time switching. Example: Concept of tmux, pip download interface.

Examples of real-time applications

```
#include <iostream>
#include <conio.h>
#include <Windows.h> // For Sleep()

void keyboardInterruptHandler() {
    if (_kbhit()) {
        char key = _getch();
        std::cout << "Key pressed: " << key << std::endl;
    }
}

int main() {
    while (true) {
        // Simulate other program logic
        Sleep(100); // Sleep for 100 milliseconds

        // Check for keyboard input through the interrupt handler
        keyboardInterruptHandler();
    }

    return 0;
}
```

The example above shows a real-time keyboard polling mechanism.

Another example in CSOPESY emulator, where it shows the main loop. A real-time application would typically have a while loop wherein the application exits the loop when an exit/terminating sequence is triggered.

```
27 int main()
28 {
29     InputManager::initialize();
30     FileSystem::initialize();
31     // FileSystem::getInstance()->test_createRandomFiles(1000);
32     // FileSystem::getInstance()->saveFileSystem();
33     FileSystem::getInstance()->loadFileSystem();
34     ConsoleManager::initialize();
35     MessageBuffer::initialize();
36     ResourceEmulator::initialize();
37     MemoryManager::initialize();
38
39     bool running = true;
40     while(running)
41     {
42         ConsoleManager::getInstance()->process();
43         ConsoleManager::getInstance()->drawConsole();
44
45         running = ConsoleManager::getInstance()->isRunning();
46     }
47
48     // MemoryManager::test_MemoryAllocation();
49
50     MemoryManager::destroy();
51     ResourceEmulator::destroy();
52     MessageBuffer::destroy();
53     ConsoleManager::destroy();
54     InputManager::destroy();
55     return 0;
56 }
```

Another example in a game application where real-time interaction is highly valued:

```
// Game Loop
void RunGameLoop(IGameEvent& gameEvent) {
    const float targetFPS = 60.0f;
    const float frameTime = 1.0f / targetFPS;

    while (true) {
        // Handle user input
        if (_kbhit()) {
            char key = _getch();
            gameEvent.OnKeyPressed(key);
        }

        // Update game logic
        gameEvent.OnUpdate(frameTime);

        // Render game state
        gameEvent.OnRender();

        // Sleep to achieve a constant frame rate
        Sleep(static_cast<DWORD>(frameTime * 1000.0f));
    }
}

int main() {
    GameEventHandler gameHandler;

    // Start the game loop
    RunGameLoop(gameHandler);

    return 0;
}
```

Examples of event-driven applications

Event-driven applications are those in which the flow of the program is determined by events such as user actions, sensor outputs, or messages from other programs

- Graphical User Interface (GUI) Applications
 - Example: A simple text editor
 - Events: Mouse clicks, keypresses, menu selections
 - Usage: When the user interacts with the GUI by clicking buttons, typing text, or choosing options from menus, events are triggered, and the application responds accordingly.

The code below shows Qt framework in C++, designed for event-driven applications:

<https://www.qt.io/product/framework>

```

1 //The code below shows Qt framework in C++, designed for event-driven applications.
2 #include <QApplication>
3 #include <QTextEdit>
4 #include <QMenuBar>
5 #include <QMenu>
6 #include <QAction>
7 #include <QFileDialog>
8
9 class TextEditor : public QTextEdit {
10     Q_OBJECT
11
12 public:
13     TextEditor(QWidget *parent = nullptr) : QTextEdit(parent) {
14         createActions();
15         createMenus();
16     }
17
18 private:
19     void newFile() {
20         clear();
21     }
22
23     void openFile() {
24         //open file
25     }
26
27     void saveFile() {
28         //save file
29     }
30
31 private:
32     void createActions() {
33         newAction = new QAction(tr("&New"), this);
34         newAction->setShortcut(QKeySequence::New);
35         connect(newAction, &QAction::triggered, this, &TextEditor::newFile);
36
37         openAction = new QAction(tr("&Open"), this);
38         openAction->setShortcut(QKeySequence::Open);
39         connect(openAction, &QAction::triggered, this, &TextEditor::openFile);
40
41         saveAction = new QAction(tr("&Save"), this);
42         saveAction->setShortcut(QKeySequence::Save);
43         connect(saveAction, &QAction::triggered, this, &TextEditor::saveFile);
44     }
45
46     void createMenus() {
47         fileMenu = menuBar()->addMenu(tr("&File"));
48         fileMenu->addAction(newAction);
49         fileMenu->addAction(openAction);
50         fileMenu->addAction(saveAction);
51     }
52
53     QMenu *fileMenu;
54     QAction *newAction;
55     QAction *openAction;
56     QAction *saveAction;
57 };
58
59 int main(int argc, char *argv[]) {
60     QApplication app(argc, argv);
61
62     TextEditor textEditor;
63     textEditor.show();
64
65     return app.exec();
66 }

```

Web Applications

- Example: An online shopping website
- Events: Button clicks, form submissions, AJAX requests
- Usage: Users interact with the website by clicking buttons to add items to their cart, submitting forms for payment, and the application responds to these events.

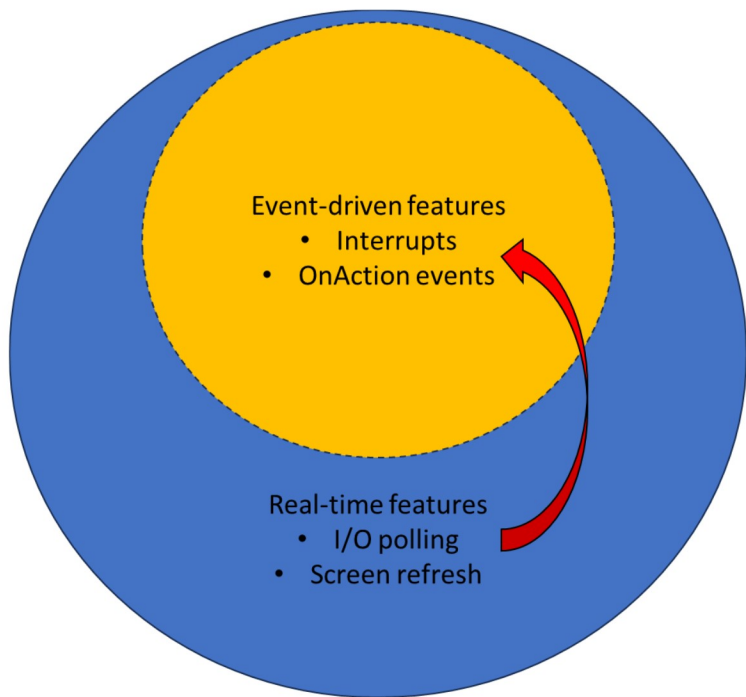
Mobile Applications:

- Example: A social media app
- Events: Taps, swipes, device orientation changes
- Usage: Users interact with the app by tapping on posts, swiping to navigate, and the application responds by displaying relevant content.

Networking Applications:

- Example: Chat application
- Events: Receiving a message, connection status changes
- Usage: When a user sends a message, or when the application receives a message from another user, events are triggered, and the application updates the chat interface.

Observation: All event-driven applications, are derived from real-time applications. All event-driven applications still require functionality that needs to run in real-time.



Family of implementation features

Keyboard polling example shown previously:
Real-time feature → keyboard polling,
Event-driven feature → IKeyboardEvent implementation

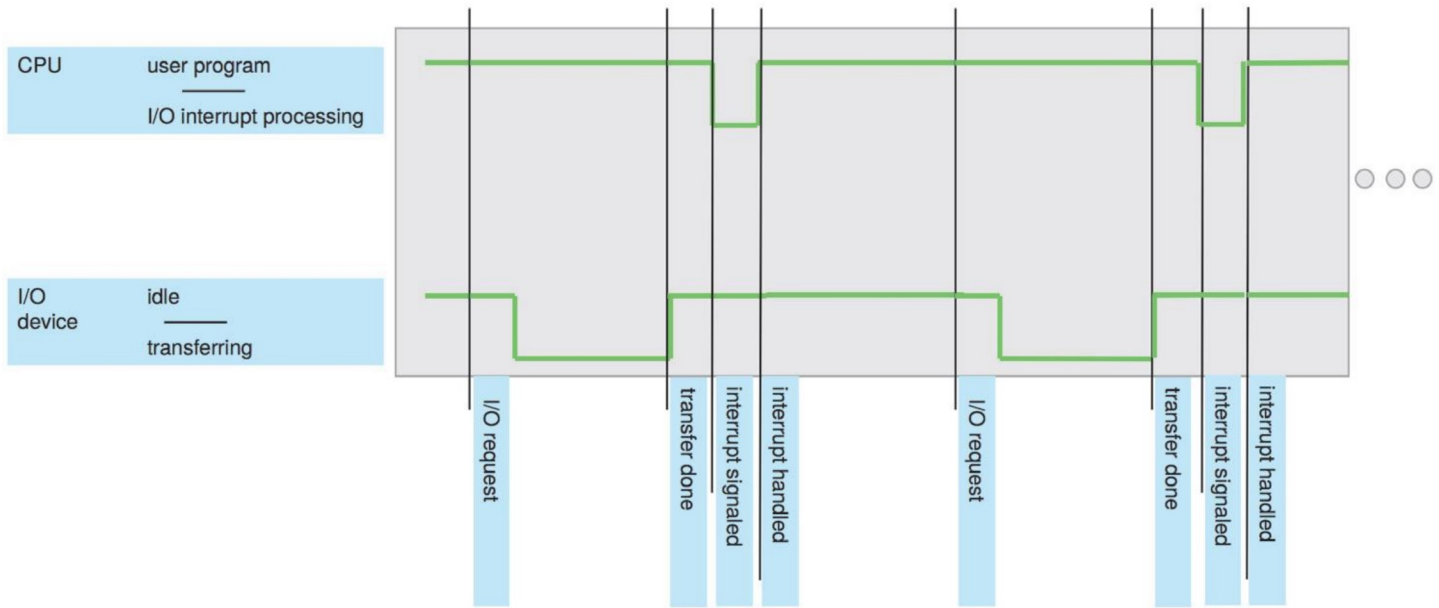
Conceptual Framework: Polling vs. Interrupts

To understand how real-time loops support event-driven functionality, we must define the two primary mechanisms for handling input/output (I/O) communication:

- **Polling (Active Checking):**
 - A mechanism where the application continuously and actively queries the status of an I/O device or buffer to see if an event has occurred.
 - In our console architecture, this is represented by the PollKeyboard function looping indefinitely and using _kbhit() to check for key presses every cycle.
 - Pros: Extremely predictable timing; perfect for real-time applications (like games or emulators) that must refresh the screen every frame regardless of input.
 - Cons: Can result in high CPU utilization when idle due to busy-waiting.
- **Interrupts (Event-Driven Notification):**
 - A mechanism where the CPU or framework is alerted only when an external event occurs, temporarily suspending normal program execution to route control to an associated handler.
 - Instead of constantly asking "Is there data?", the system waits silently until it is interrupted by a trigger.
 - Pros: Highly efficient; dramatically reduces CPU wakeups when the application is idle.
 - Cons: Can introduce execution latency or frame-rate stutter in real-time graphic render pipelines if an interrupt handler stalls the loop.

- At the user application level, we mostly **deal with polling**. Interrupts are handled by event callbacks.
- Event callbacks act like a “queue” of interrupt events (e.g., very fast key presses happening before the next `_kbhit()` event).
- **Non-blocking vs blocking I/O**
 - Developers / APIs often have two ways to process I/O, which is either non-blocking or blocking.
 - `_kbhit()` is non-blocking. `getch()` is blocking.

Illustration of I/O interrupt processing, on a `_kbhit()` event.



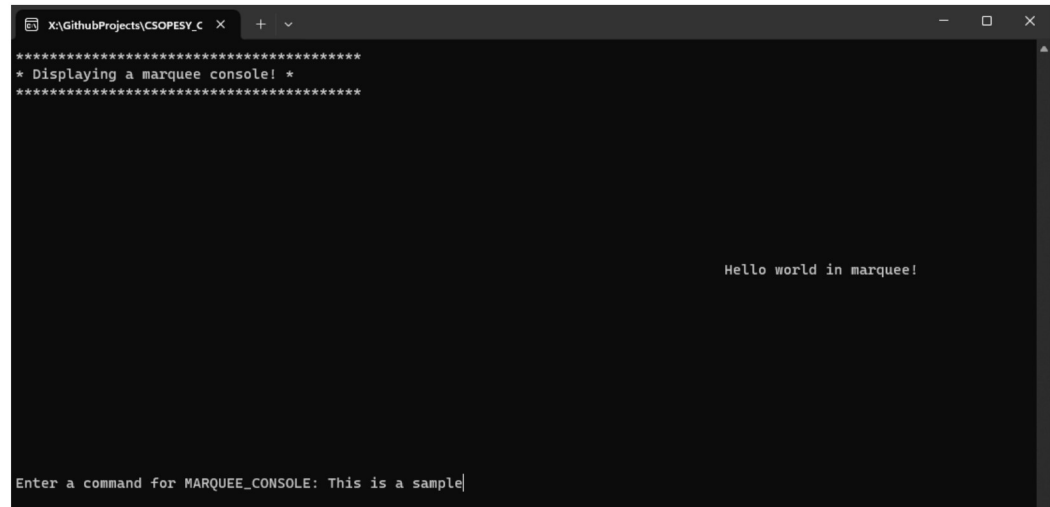
```
95 //Here is an example of a simple C++ application that combines real-time and event-driven handling:
96 #include <conio.h> // For _kbhit() and _getch()
97
98 // Interface for Keyboard Events
99 class IKeyboardEvent {
100 public:
101     virtual void OnKeyDown(char key) = 0;
102     virtual void OnKeyUp(char key) = 0;
103 };
104 // Implementation of IKeyboardEvent
105 class KeyboardEventHandler : public IKeyboardEvent {
106 public:
107     void OnKeyDown(char key) override {
108         std::cout << "Key Down: " << key << std::endl;
109     }
110
111     void OnKeyUp(char key) override {
112         std::cout << "Key Up: " << key << std::endl;
113     }
114 };
115 // Keyboard Polling
116 void PollKeyboard(IKeyboardEvent& keyboardEvent) {
117     while (true) {
118         if (_kbhit()) { // Check if a key has been pressed
119             char key = _getch(); // Get the pressed key
120
121             // Check if it's a key down or key up event
122             if (GetAsyncKeyState(key) & 0x8000) {
123                 keyboardEvent.OnKeyDown(key);
124             } else {
125                 keyboardEvent.OnKeyUp(key);
126             }
127         }
128
129         // Other program logic can continue here
130     }
131 }
132
133 int main() {
134     KeyboardEventHandler keyboardHandler;
135     // Start keyboard polling
136     PollKeyboard(keyboardHandler);
137     return 0;
138 }
```

- The IKeyboardEvent class defines an interface with OnKeyDown and OnKeyUp virtual functions.
- The KeyboardEventHandler class implements this interface, printing messages to the console when a key is pressed or released.
- The PollKeyboard function continuously checks for key presses using _kbhit() and retrieves the key using _getch().
- Depending on the state of the key using GetAsyncKeyState, it calls the appropriate event handler.

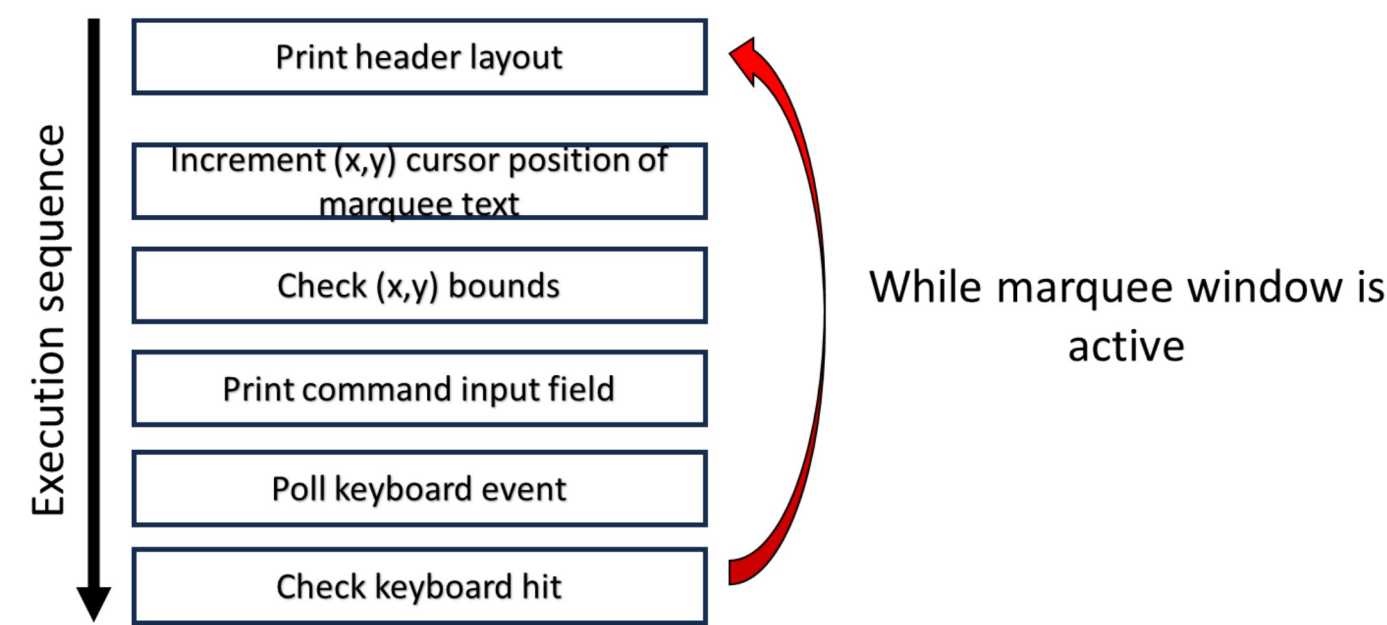
IMPLEMENTING BASIC REAL-TIME APPLICATION USING KEYBOARD POLLING

- The goal is to support **real-time handling of I/O** while performing some CPU-based logic, such as a simple animation on the screen. This demonstrates the interleaving of I/O with CPU-based logic.
- Thus, for every cycle, we must have mechanisms to refresh the screen per frame, while polling for I/O events.

We shall demonstrate this with a marquee console → an animated text that moves across the screen per frame while still allowing the user to type in text.



General flow of the marquee console window is shown below:

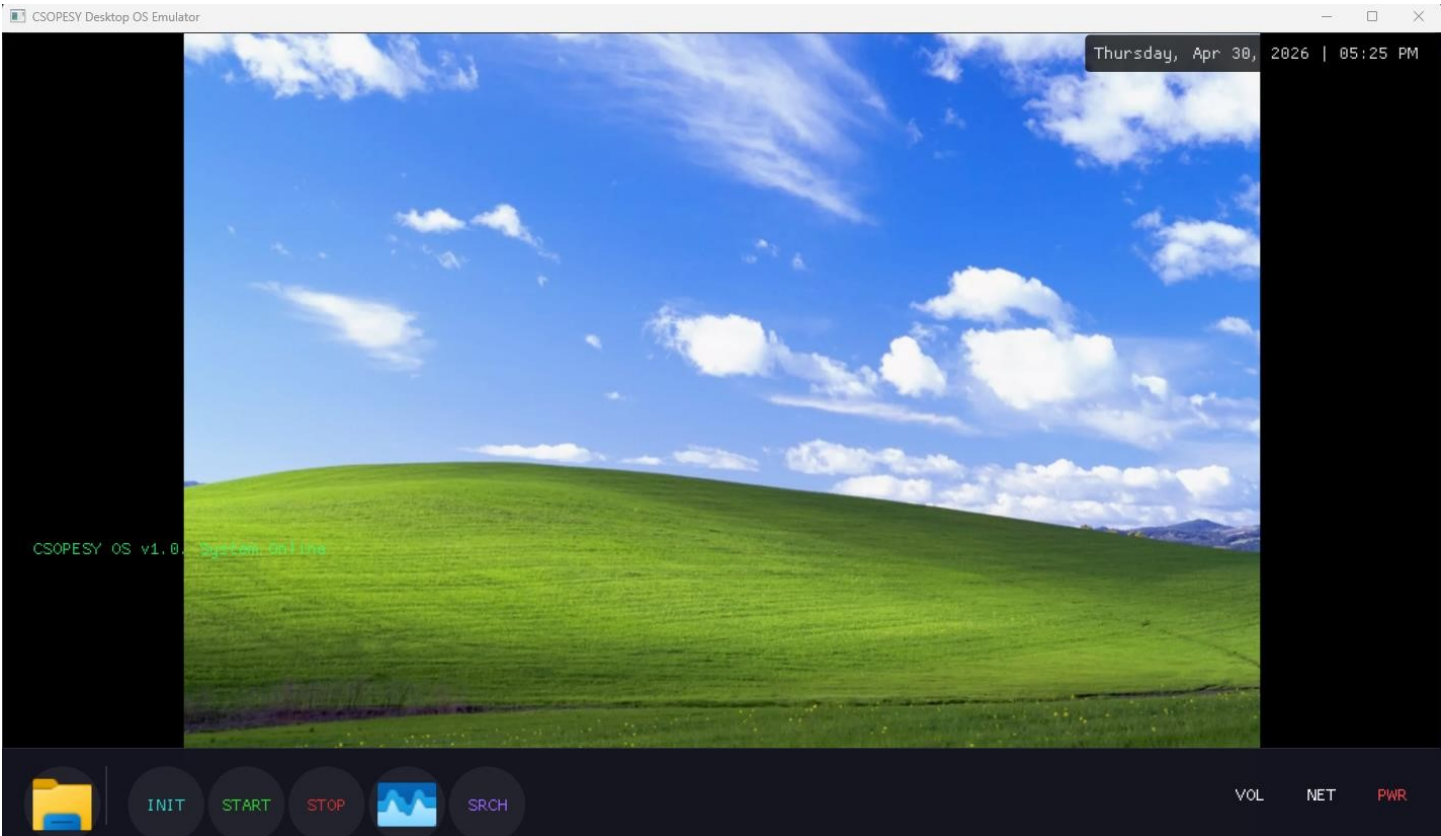
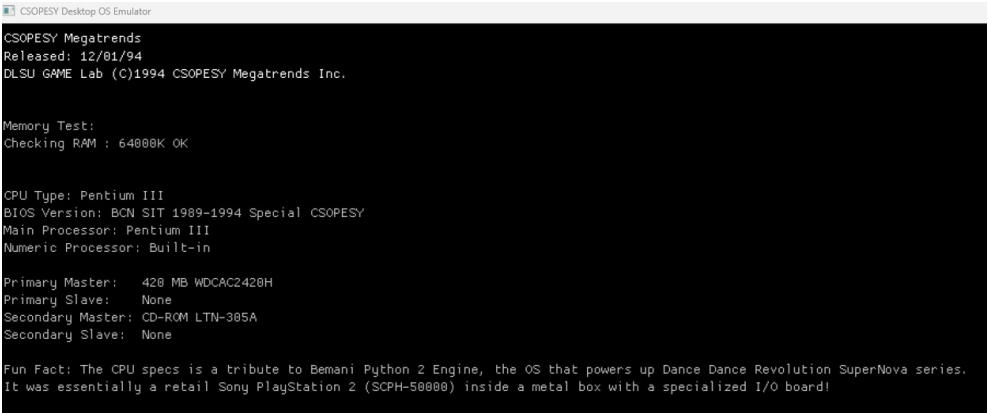


To create a customized console layout, we maximize the functionalities provided by the Windows Console API: <https://learn.microsoft.com/en-us/windows/console/console-functions>

```
114 void ConsoleManager::setCursorPosition(int posX, int posY) const
115 {
116     COORD coord;
117     coord.X = posX;
118     coord.Y = posY;
119     SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), coord);
120 }
```

CREATING REAL-TIME INTERFACES / DESKTOP-STYLE OS

- An extension of the basic real-time application, by using an immediate-mode library available.
- Easiest/low-friction concept: Use an open-source immediate-mode library such as dear ImGui: <https://github.com/ocornut/imgui>





Why ImGui for OS?

- Simplicity: No complex widget trees or event hierarchies to manage
- Real-time focus: Perfect for systems that need per-frame updates (like OS emulators)
- Educational value: Students see the render loop explicitly, connecting to OS scheduling concepts
- Industry relevance: Used in game engines, real-time developer tools, and embedded systems.

Connection to Previous Weeks:

In Week 2, we covered the **five-phase OS loading sequence** (Bootstrapping → Kernel Init → System Services → Main Loop → Shutdown). Section 2.0 of this handout maps each of those phases to specific CSOPESY source files and functions.

In Weeks 4-5, we covered: • Console-based keyboard polling using `_kbhit()` and `_getch()` • Real-time vs event-driven application design • The `AConsole` class and `ConsoleManager` for multi-window console handling

In Weeks 6-7, we extend these concepts to: • Desktop GUI with windows, taskbars, and graphical widgets • GLFW-based event polling (`glfwPollEvents()`) • `ImGui`'s immediate-mode rendering paradigm • Real-time data visualization (performance graphs, process tables)

2. From Console to Desktop: Evolution of the CSOPESY Emulator
2.0 Recap: A Typical Sequence of How an OS is Loaded and Run (Week 2 Review)

This subsection bridges Week 2's overview of the CSOPESY OS emulator with the desktop GUI concepts introduced in Weeks 6-7. Remind students that everything they are building — the boot window, the desktop, the taskbar — maps directly to the five-phase OS loading sequence they studied earlier. In Week 2, we established the **canonical five-phase sequence** by which any operating system — real or emulated — is loaded and begins running. This sequence is not just theoretical: every component you will build in Weeks 6-7 corresponds to a specific phase in this lifecycle.

The Five Phases mapped to UI sequence

Phase	Description	CSOPESY Emulator Equivalent
1. Bootstrapping	Low-level hardware setup; load kernel into memory	GUIApplication initializes GLFW + OpenGL; loads fonts and DPI config
2. Kernel Initialization	Set up data structures, interrupt handlers, memory management	UIManager singleton created; UIConfig calculates DPI scale; window registry initialized
3. Start System Services	Launch essential services, daemons, background processes	Desktop, Taskbar, TaskManagerUI windows registered and made visible

Phase	Description	CSOPESY Emulator Equivalent
4. Enter Main Loop	Continuously handle interrupts, dispatch processes, handle I/O	GUIApplication::run() → glfwPollEvents() → ImGui::NewFrame() → renderFrame() → glfwSwapBuffers()
5. Shutdown and Cleanup	Gracefully terminate processes, release memory, halt/reboot	GLFW window close callback → ImGui::DestroyContext() → glfwTerminate()

Phase 1 → Bootstrapping in CSOPESY

In a real OS, bootstrapping performs hardware initialization and loads the kernel. In CSOPESY, the equivalent is **GLFW and OpenGL initialization** inside GUIApplication:

```
// Phase 1: Bootstrapping – hardware/window system setup
glfwInit();
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
GLFWwindow* window = glfwCreateWindow(width, height, "CSOPESY", nullptr, nullptr);
glfwMakeContextCurrent(window);

// Load OpenGL function pointers (equivalent to loading kernel into memory)
gladLoadGLLoader((GLADloadproc)glfwGetProcAddress);

// Initialize Dear ImGui (equivalent to kernel entry point)
IMGUI_CHECKVERSION();
ImGui::CreateContext();
ImGui_ImplGlfw_InitForOpenGL(window, true);
ImGui_ImplOpenGL3_Init("#version 330");
```

DEMO: Open GUIApplication.cpp. Trace the constructor and initialize() method. Map each function call to the bootstrapping steps listed in Week 2 (hardware init → memory init → load kernel).

Phase 2 → Kernel Initialization in CSOPESY

Kernel initialization sets up data structures and memory management. In CSOPESY, this is **UIConfig and UIManager initialization** — the "kernel data structures" of the GUI subsystem:

```
// Phase 2: Kernel Initialization – set up data structures
UIConfig::getInstance()->calculateScaling(monitorWidth, monitorHeight);
UIManager::getInstance()->initialize();

// Set up "interrupt handlers" – GLFW callbacks
glfwSetKeyCallback(window, keyCallback);
glfwSetMouseButtonCallback(window, mouseButtonCallback);
glfwSetScrollCallback(window, scrollCallback);
```

Observation: UIManager is a **singleton registry** — analogous to the process table in a real kernel. Just as the OS kernel keeps a table of all running processes, UIManager keeps a registry of all active GUI windows.

Phase 3 → System Services in CSOPESY

The OS launches essential daemons and background services. In CSOPESY, this is **registering and showing the core GUI windows**:

```
// Phase 3: Start System Services – launch core windows
auto desktop = std::make_shared<Desktop>();
auto taskbar = std::make_shared<Taskbar>();
auto taskMgr = std::make_shared<TaskManagerUI>();
auto bootWindow = std::make_shared<BIOSBootWindow>();

UIManager::getInstance()->registerWindow("desktop", desktop);
UIManager::getInstance()->registerWindow("taskbar", taskbar);
UIManager::getInstance()->registerWindow("taskManager", taskMgr);
UIManager::getInstance()->registerWindow("boot", bootWindow);

UIManager::getInstance()->showWindow("boot"); // Start with boot sequence
```

Observation (from Week 2): *"These are processes that are persistent in the OS — always alive / periodically invoking functions."* The Desktop and Taskbar windows are exactly this: **persistent system services** that render every frame, just like OS daemons that are always running in the background.

Phase 4 → Main Loop in CSOPESY

The most critical phase: the OS enters an **infinite loop** to handle events, dispatch processes, and manage I/O. In CSOPESY, this is the **IMGUI render loop**:

```
// Phase 4: Enter Main Loop - handle events, dispatch, I/O
while (!glfwWindowShouldClose(window)) {
    // Handle interrupts / poll for input events
    glfwPollEvents();

    // Dispatch "processes" - render all active windows
    ImGui::NewFrame();
    UIManager::getInstance()->renderAllWindows();
    ImGui::Render();

    // Flush output - present the rendered frame
    ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());
    glfwSwapBuffers(window);
}
```

In the console-based emulator, the main loop used `_kbhit()` to poll for keyboard input. In the desktop GUI, `glfwPollEvents()` takes over this role — it polls **all** input devices (keyboard, mouse, window resize, etc.) and queues the events for ImGui to process. The polling *philosophy* is identical; only the mechanism changes.

DEMO: Place a breakpoint inside the `while` loop. Show students that this loop runs approximately 60 times per second (60 FPS). Ask: *"What happens if one window's `draw()` function takes 100ms? How does that affect the entire system?"*

Phase 5 → Shutdown in CSOPESY

When the user closes the window, the emulator must clean up — just as a real OS gracefully terminates processes and releases memory:

```
// Phase 5: Shutdown and Cleanup
ImGui_ImplOpenGL3_Shutdown();
ImGui_ImplGlfw_Shutdown();
ImGui::DestroyContext();
glfwDestroyWindow(window);
glfwTerminate();
```

Observation: Notice that cleanup is done in **reverse initialization order** — ImGui is shut down before GLFW, just as user-space processes are terminated before the kernel shuts down in a real OS.

Mapping the BIOSBootWindow to Real BIOS/UEFI Boot

The BIOSBootWindow is a deliberate pedagogical tool: it *simulates* the visual experience of a real BIOS/UEFI boot sequence. Compare the phases:

Real BIOS/UEFI Phase	BIOSBootWindow Phase	What the Student Sees
POST (Power-On Self Test)	BIOS_HEADER	ASCII art logo, version info
RAM check	RAM_CHECK	Animated memory scan progress bar
Hardware enumeration	HARDWARE_INFO	List of detected (simulated) devices
Boot settings display	BIOS_SETTINGS	Configuration summary
OS handoff	LOADING_OS	"Loading CSOPESY OS..." spinner
Kernel running	COMPLETE	Transition to Desktop

Activity: Ask students to open BIOSBootWindow.cpp and identify which BootPhase enum value corresponds to each of the five Week 2 OS loading phases. This is a quick 5-minute in-class mapping exercise.

ImGui Fundamentals: Immediate Mode vs Retained Mode

What is Immediate Mode GUI?

Definition: An immediate-mode GUI rebuilds the entire interface **every frame** from scratch, with minimal retained state between frames [1].

Contrast with Retained Mode:

Aspect	Retained Mode (e.g., Qt, WPF)	Immediate Mode (e.g., Dear ImGui)
Widget storage	Widgets stored in persistent tree/hierarchy	No persistent widget objects
State management	Framework manages widget state	Application manages state
Update model	Invalidate/repaint specific regions	Rebuild entire UI each frame
Memory overhead	Higher (full widget tree)	Lower (minimal retained state)
Code complexity	Event handlers, callbacks, signals/slots	Direct procedural code in render loop
Best for	Complex, persistent UIs (desktop apps)	Real-time tools, debug overlays, editors

Why Immediate Mode for CSOPESY?

Advantages for OS emulation:

- Simplicity:** Students write straightforward procedural code, not complex event hierarchies
- Real-time focus:** Natural fit for systems that update every frame (like OS schedulers)
- Dynamic layouts:** Easy to show/hide windows based on process state
- Educational clarity:** The render loop is explicit, connecting to OS scheduling concepts

The ImGui Philosophy

Core principle: "If you want to display a button, just call `Button()` every frame."

```
// Retained mode (Qt-style pseudocode)
Button* myButton = new Button("Click Me");
myButton->setOnClick([]{ doSomething(); });
layout->addWidget(myButton);

// Immediate mode (ImGui)
void renderFrame() {
    if (ImGui::Button("Click Me")) {
        doSomething();
    }
}
```

Observation: In immediate mode, the button doesn't "exist" between frames. The `Button()` function returns `true` if clicked **this frame**, and you handle it immediately.

The ImGui Render Loop: Architecture and Implementation

The Main Application Loop

Key file: ``GUIApplication.cpp``

The heart of the ImGui application is the **render loop**, which runs continuously until the user exits:

```
void GUIApplication::run() {
    while (!glfwWindowShouldClose(window)) {
        // 1. Poll input events (keyboard, mouse, window events)
        glfwPollEvents();

        // 2. Start new ImGui frame
        ImGui_ImplOpenGL3_NewFrame();
        ImGui_ImplGlfw_NewFrame();
        ImGui::NewFrame();

        // 3. Update application logic
        updateLogic();

        // 4. Render all UI components
        renderFrame();

        // 5. Finalize ImGui rendering
```

```
ImGui::Render();

// 6. Clear screen and render ImGui draw data
int display_w, display_h;
glfwGetFramebufferSize(window, &display_w, &display_h);
glViewport(0, 0, display_w, display_h);
glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
glClear(GL_COLOR_BUFFER_BIT);
ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());

// 7. Swap front and back buffers (double buffering)
glfwSwapBuffers(window);
}
```

Breaking Down the Render Loop

Step 1: `glfwPollEvents()`

- Polls the OS for input events (keyboard, mouse, window resize, etc.)
- Non-blocking: returns immediately with all pending events
- GLFW translates OS-specific events into cross-platform callbacks
- ImGui's GLFW backend automatically captures these events via registered callbacks

Connection to OS design: This is analogous to the kernel's interrupt handler checking the interrupt queue. Modern OSes use **event-driven I/O** at the kernel level to reduce CPU wakeups [4].

Step 2: `ImGui::NewFrame()`

- Prepares ImGui for a new frame of rendering
- Resets internal state (draw lists, input state, etc.)
- Captures input state from `ImGuiIO` (mouse position, key states, etc.)

Step 3: `updateLogic()`

- Application-specific logic (process scheduling, memory management, etc.)
- Updates data structures that will be displayed in the UI
- In CSOPESY: updates process states, CPU utilization, memory usage

Step 4: `renderFrame()`

- Calls all UI rendering functions (Desktop, Taskbar, Windows, etc.)
- Each component calls ImGui functions to declare its UI
- ImGui builds internal draw lists (vertices, indices, textures)

Step 5: `ImGui::Render()`

- Finalizes ImGui's internal draw data
- Converts high-level widget calls into low-level rendering commands

Step 6: OpenGL Rendering

- Clears the framebuffer
- Renders ImGui's draw data using OpenGL
- All UI is drawn to the back buffer

Step 7: `glfwSwapBuffers()`

- Swaps front and back buffers (double buffering)
- Presents the rendered frame to the screen
- Typically synchronized with monitor refresh rate (VSync)

Polling vs Event-Driven in GLFW

GLFW offers two input models:

1. **Polling** (`glfwPollEvents()`):
 - Checks for events once per frame
 - Non-blocking
 - Suitable for real-time applications (games, simulations)
 - Used in CSOPESY emulator
2. **Waiting** (`glfwWaitEvents()`):
 - Blocks until an event occurs
 - Reduces CPU usage when idle
 - Suitable for event-driven applications (text editors, IDEs)

DEMO: Modify `GUIApplication.cpp` to use `glfwWaitEvents()` instead of `glfwPollEvents()`. Show how the UI freezes until user input occurs. Discuss trade-offs.

Building Desktop UI Components

The Desktop: Full-Screen Background

Key file: `Desktop.cpp`

The Desktop is the foundation of the GUI, rendered as a full-screen ImGui window:

```
void Desktop::draw() {
    // Create full-screen window
    ImGui::SetNextWindowPos(ImVec2(0, 0));
    ImGui::SetNextWindowSize(ImGui::GetIO().DisplaySize);

    ImGui::Begin("Desktop", nullptr,
        ImGuiWindowFlags_NoTitleBar |
        ImGuiWindowFlags_NoResize |
        ImGuiWindowFlags_NoMove |
        ImGuiWindowFlags_NoCollapse |
        ImGuiWindowFlags_NoBringToFrontOnFocus);

    // Render wallpaper
    if (wallpaperTexture) {
        ImGui::GetWindowDrawList()->AddImage(
            (void*)(intptr_t)wallpaperTexture,
            ImVec2(0, 0),
            ImGui::GetIO().DisplaySize
        );
    }

    // Render real-time clock
    drawClock();

    ImGui::End();
}

void Desktop::drawClock() {
    auto now = std::chrono::system_clock::now();
    auto time = std::chrono::system_clock::to_time_t(now);
    struct tm timeinfo;
    localtime_s(&timeinfo, &time);

    char buffer[64];
    strftime(buffer, sizeof(buffer), "%I:%M:%S %p", &timeinfo);

    ImVec2 textSize = ImGui::CalcTextSize(buffer);
    ImVec2 pos = ImVec2(ImGui::GetIO().DisplaySize.x - textSize.x - 20, 20);

    ImGui::SetCursorPos(pos);
    ImGui::TextColored(ImVec4(1, 1, 1, 1), "%s", buffer);
}
```

- Key techniques:**
- `SetNextWindowPos/Size()`: Position window before `Begin()`
 - Window flags: Remove title bar, disable resizing/moving
 - `GetWindowDrawList()`: Access low-level drawing API for custom rendering
 - `CalcTextSize()`: Measure text dimensions for positioning

DEMO: Show how removing `ImGuiWindowFlags_NoTitleBar` makes the desktop a movable window. Discuss why these flags are necessary.

The Taskbar: Bottom Panel with Icons

Key file: `Taskbar.cpp`

The Taskbar is a horizontal panel at the bottom of the screen with clickable icons:

```
void Taskbar::draw() {
    ImVec2 displaySize = ImGui::GetIO().DisplaySize;
    float taskbarHeight = 60.0f * UIConfig::getScaleFactor();

    // Position at bottom of screen
    ImGui::SetNextWindowPos(ImVec2(0, displaySize.y - taskbarHeight));
    ImGui::SetNextWindowSize(ImVec2(displaySize.x, taskbarHeight));

    ImGui::Begin("Taskbar", nullptr,
        ImGuiWindowFlags_NoTitleBar |
        ImGuiWindowFlags_NoResize |
        ImGuiWindowFlags_NoMove |
        ImGuiWindowFlags_NoScrollbar);
}
```

```

// Render icons
ImGui::PushStyleVar(ImGuiStyleVar_ItemSpacing, ImVec2(10, 0));

for (auto& icon : taskbarIcons) {
    if (icon.texture) {
        if (ImGui::ImageButton((void*)(intptr_t)icon.texture,
                                ImVec2(40, 40) * UIConfig::getScaleFactor())) {
            icon.onClick();
        }
        if (ImGui::IsItemHovered()) {
            ImGui::SetTooltip("%s", icon.name.c_str());
        }
    }
    ImGui::SameLine();
}

ImGui::PopStyleVar();

// System tray (right side)
drawSystemTray();

ImGui::End();
}

void Taskbar::drawSystemTray() {
    ImGui::SameLine(ImGui::GetWindowWidth() - 200);

    // CPU usage
    ImGui::Text("CPU: %.1f%%", getCPUUsage());
    ImGui::SameLine();

    // Memory usage
    ImGui::Text("MEM: %.1f%%", getMemoryUsage());
    ImGui::SameLine();

    // Clock
    ImGui::Text("%s", getCurrentTime().c_str());
}

```

Key techniques:

- `ImageButton()`: Clickable image widget
- `IsItemHovered()`: Detect mouse hover for tooltips
- `SameLine()`: Place widgets horizontally
- `PushStyleVar/PopStyleVar()`: Temporarily modify spacing
- `GetWindowWidth()`: Calculate right-aligned positions

Observation: The taskbar uses **DPI scaling** (`UIConfig::getScaleFactor()`) to ensure consistent size across different monitor resolutions. We'll cover this in detail in Section 9.

Real-Time Data Visualization

Task Manager: Performance Graphs

Key file: `TaskManagerUI.cpp`

The Task Manager displays real-time performance data using ImGui's plotting widgets:

```

void TaskManagerUI::draw() {
    if (!this->beginWindow()) return;

    // Tab bar for different views
    if (ImGui::BeginTabBar("TaskManagerTabs")) {
        if (ImGui::BeginTabItem("Performance")) {
            drawPerformanceTab();
            ImGui::EndTabItem();
        }
        if (ImGui::BeginTabItem("Processes")) {
            drawProcessesTab();
            ImGui::EndTabItem();
        }
        ImGui::EndTabBar();
    }

    this->endWindow();
}

```

```
void TaskManagerUI::drawPerformanceTab() {
    // CPU usage graph
    ImGui::Text("CPU Usage");
    ImGui::PlotLines("##CPU", cpuHistory.data(), cpuHistory.size(),
        0, nullptr, 0.0f, 100.0f, ImVec2(0, 80));

    // Memory usage graph
    ImGui::Text("Memory Usage");
    ImGui::PlotLines("##Memory", memoryHistory.data(), memoryHistory.size(),
        0, nullptr, 0.0f, 100.0f, ImVec2(0, 80));

    // Update history buffers
    updatePerformanceData();
}

void TaskManagerUI::updatePerformanceData() {
    // Shift history buffer
    for (int i = 0; i < cpuHistory.size() - 1; i++) {
        cpuHistory[i] = cpuHistory[i + 1];
    }

    // Add new data point
    cpuHistory[cpuHistory.size() - 1] = getCurrentCPUUsage();

    // Same for memory
    for (int i = 0; i < memoryHistory.size() - 1; i++) {
        memoryHistory[i] = memoryHistory[i + 1];
    }
    memoryHistory[memoryHistory.size() - 1] = getCurrentMemoryUsage();
}
```

Key techniques:

- PlotLines(): Render line graph from array of floats
- BeginTabBar/BeginTabItem(): Create tabbed interface
- Circular buffer pattern: Shift old data, append new data

Connection to real-time systems: This is a classic **sliding window** visualization, common in system monitors (Windows Task Manager, macOS Activity Monitor, Linux htop).

Process Table: Sortable Columns

```
void TaskManagerUI::drawProcessesTab() {
    if (ImGui::BeginTable("ProcessTable", 5,
        ImGuiTableFlags_Sortable |
        ImGuiTableFlags_Resizable |
        ImGuiTableFlags_Borders |
        ImGuiTableFlags_RowBg)) {

        // Setup columns
        ImGui::TableSetupColumn("PID", ImGuiTableColumnFlags_DefaultSort);
        ImGui::TableSetupColumn("Name");
        ImGui::TableSetupColumn("State");
        ImGui::TableSetupColumn("CPU %");
        ImGui::TableSetupColumn("Memory");
        ImGui::TableHeadersRow();

        // Get sort specs
        if (ImGuiTableSortSpecs* sortSpecs = ImGui::TableGetSortSpecs()) {
            if (sortSpecs->SpecsDirty) {
                sortProcesses(sortSpecs);
                sortSpecs->SpecsDirty = false;
            }
        }

        // Render rows
        for (const auto& process : processes) {
            ImGui::TableNextRow();

            ImGui::TableNextColumn();
            ImGui::Text("%d", process.pid);

            ImGui::TableNextColumn();
            ImGui::Text("%s", process.name.c_str());

            ImGui::TableNextColumn();
            ImVec4 stateColor = getStateColor(process.state);
            ImGui::TextColored(stateColor, "%s", getStateName(process.state));

            ImGui::TableNextColumn();
            ImGui::Text("%.1f%%", process.cpuUsage);
        }
    }
}
```

```

        ImGui::TableNextColumn();
        ImGui::Text("%zu KB", process.memoryUsage);
    }

    ImGui::EndTable();
}

}

ImVec4 TaskManagerUI::getStateColor(ProcessState state) {
    switch (state) {
        case ProcessState::RUNNING:
            return ImVec4(0.0f, 1.0f, 0.0f, 1.0f); // Green
        case ProcessState::READY:
            return ImVec4(1.0f, 1.0f, 0.0f, 1.0f); // Yellow
        case ProcessState::WAITING:
            return ImVec4(1.0f, 0.5f, 0.0f, 1.0f); // Orange
        case ProcessState::TERMINATED:
            return ImVec4(0.5f, 0.5f, 0.5f, 1.0f); // Gray
        default:
            return ImVec4(1.0f, 1.0f, 1.0f, 1.0f); // White
    }
}
}

```

Key techniques:

- `BeginTable()`: Create multi-column table with sorting/resizing
- `TableSetupColumn()`: Define column headers and properties
- `TableGetSortSpecs()`: Query user's sort preferences
- `TextColored()`: Color-code process states for visual clarity

DEMO: Show the Task Manager in action. Click column headers to sort. Discuss how this mirrors Windows Task Manager's design.

8. Window Management and State

The AWindow Base Class

Key file: ``AWindow.cpp``

All windows in the CSOPESY emulator inherit from AWindow, which provides common functionality:

```

class AWindow {
public:
    AWindow(const std::string& name) : windowName(name), isVisible(false) {}
    virtual ~AWindow() = default;

    virtual void draw() = 0; // Pure virtual: subclasses must implement

    void show() { isVisible = true; }
    void hide() { isVisible = false; }
    bool isShown() const { return isVisible; }

protected:
    bool beginWindow() {
        if (!isVisible) return false;

        ImGui::Begin(windowName.c_str(), &isVisible);
        return true;
    }

    void endWindow() {
        ImGui::End();
    }

    std::string windowName;
    bool isVisible;
};

```

Key design patterns:

- **Template Method:** `draw()` is pure virtual; subclasses implement specific rendering
- **RAII-style:** `beginWindow()` / `endWindow()` pair ensures `ImGui::Begin()` / `ImGui::End()` are balanced
- **Visibility management:** `isVisible` flag controls whether window is rendered

Observation: The `&isVisible` parameter in `ImGui::Begin()` creates a close button (X) that automatically sets `isVisible = false` when clicked.

The UIManager: Window Registry
Key file: `UIManager.cpp`

The UIManager maintains a registry of all windows and handles their lifecycle:

```
class UIManager {
public:
    static UIManager& getInstance() {
        static UIManager instance;
        return instance;
    }

    void registerWindow(const std::string& name, std::shared_ptr<AWindow> window) {
        windows[name] = window;
    }

    void showWindow(const std::string& name) {
        if (windows.find(name) != windows.end()) {
            windows[name]->show();
        }
    }

    void hideWindow(const std::string& name) {
        if (windows.find(name) != windows.end()) {
            windows[name]->hide();
        }
    }

    void updateAllWindows() {
        for (auto& [name, window] : windows) {
            if (window->isShown()) {
                window->update(); // Update logic (if needed)
            }
        }
    }

    void renderAllWindows() {
        for (auto& [name, window] : windows) {
            if (window->isShown()) {
                window->draw();
            }
        }
    }

private:
    std::map<std::string, std::shared_ptr<AWindow>> windows;
};
```

Usage in main render loop:

```
void GUIApplication::renderFrame() {
    desktop->draw();
    taskbar->draw();
    UIManager::getInstance().renderAllWindows();
}
```

- Key design patterns:**
- **Singleton:** Ensures single global registry
 - **Registry pattern:** Central lookup for windows by name
 - **Separation of concerns:** UIManager handles lifecycle, windows handle rendering

- Connection to OS design:** This mirrors how real OSes manage windows:
- Windows OS: Desktop Window Manager (DWM) maintains window list
 - macOS: WindowServer manages window hierarchy
 - Linux: X11/Wayland compositor tracks window state

9. DPI Scaling and Responsive Design
The Problem: High-DPI Displays

- Modern monitors have varying pixel densities:
- Standard HD: 1920×1080 (96 DPI)
 - 4K: 3840×2160 (192 DPI)
 - Retina: 2880×1800 (220 DPI)

Without DPI scaling: UI elements appear tiny on high-DPI displays.

The Solution: UIConfig

Key file: `UIConfig.cpp`

```
class UIConfig {
public:
    static void initialize() {
        // Get primary monitor
        GLFWmonitor* monitor = glfwGetPrimaryMonitor();
        const GLFWvidmode* mode = glfwGetVideoMode(monitor);

        // Calculate scale factor based on resolution
        float baseWidth = 1920.0f;
        float currentWidth = static_cast<float>(mode->width);
        scaleFactor = currentWidth / baseWidth;

        // Clamp to reasonable range
        if (scaleFactor < 1.0f) scaleFactor = 1.0f;
        if (scaleFactor > 2.0f) scaleFactor = 2.0f;
    }

    static float getScaleFactor() {
        return scaleFactor;
    }

    static ImVec2 scale(ImVec2 size) {
        return ImVec2(size.x * scaleFactor, size.y * scaleFactor);
    }

private:
    static float scaleFactor;
};

float UIConfig::scaleFactor = 1.0f;
```

Usage in UI components:

```
// Taskbar.cpp
float taskbarHeight = 60.0f * UIConfig::getScaleFactor();

// ProcessWindow.cpp
ImVec2 windowSize = UIConfig::scale(ImVec2(600, 400));
ImGui::SetNextWindowSize(windowSize, ImGuiCond_FirstUseEver);
```

ImGui's built-in DPI support:

```
void GUIApplication::setupImGui() {
    ImGuiIO& io = ImGui::GetIO();

    // Enable DPI awareness
    io.ConfigFlags |= ImGuiConfigFlags_DpiEnableScaleFonts;
    io.ConfigFlags |= ImGuiConfigFlags_DpiEnableScaleViewports;

    // Load font with appropriate size
    float fontSize = 16.0f * UIConfig::getScaleFactor();
    io.Fonts->AddFontFromFileTTF("fonts/Roboto-Regular.ttf", fontSize);
}
```

DEMO: Run the emulator on different monitor resolutions (or simulate with window resizing). Show how UI elements scale proportionally.

Practice Exercises and Activities

Exercise 1: Custom Desktop Widget

Objective: Create a custom desktop widget that displays real-time system information.

Requirements:

- 1. Inherit from AWindow
- 2. Display at least 3 real-time metrics (e.g., CPU usage, memory usage, network activity)
- 3. Update every frame
- 4. Position in a fixed location on the desktop (e.g., top-right corner)
- 5. Use color-coding to indicate status (green = normal, yellow = warning, red = critical)

Starter code:

```
class SystemMonitorWidget : public AWindow {
public:
    SystemMonitorWidget() : AWindow("System Monitor") {}

    void draw() override {
        if (!this->beginWindow()) return;

        // TODO: Display CPU usage
        // TODO: Display memory usage
        // TODO: Display network activity
        // TODO: Color-code based on thresholds

        this->endWindow();
    }
};
```

Discussion questions:

- How often should you update the metrics? Every frame, or less frequently?
- What are the performance implications of querying system metrics every frame?
- How would you implement a "minimize to taskbar" feature?

Exercise 2: Animated Boot Sequence

Objective: Extend BIOSBootWindow with additional boot phases and animations.

Requirements:

1. Add at least 2 new boot phases (e.g., "Loading Drivers", "Starting Services")
2. Each phase should have a progress bar that fills over time
3. Display a list of items being loaded (e.g., "Loading driver: USB Controller")
4. Use ImGui::GetTime() for timing
5. Transition to desktop when complete

Starter code:

```
enum class BootPhase {
    BIOS_LOGO,
    MEMORY_CHECK,
    DEVICE_INIT,
    LOADING_DRIVERS,    // New phase
    STARTING_SERVICES,  // New phase
    BOOT_COMPLETE
};

void BIOSBootWindow::drawLoadingDrivers() {
    ImGui::Text("Loading Drivers...");

    // TODO: Display list of drivers
    // TODO: Show progress bar
    // TODO: Advance to next phase when complete
}
```

Discussion questions:

- How does this boot sequence compare to a real OS boot (e.g., Windows, Linux)?
- What information would a real BIOS/UEFI display during boot?
- How would you implement a "skip boot animation" feature?

Exercise 3: Process Window Enhancements

Objective: Add interactive features to `ProcessWindow`.

Requirements:

1. Add a "Pause/Resume" button that changes process state
2. Add a "Kill Process" button with confirmation dialog
3. Display a real-time progress bar for process completion
4. Add a "Priority" dropdown to change process scheduling priority
5. Log all user actions to the command log

Starter code:

```
void ProcessWindow::draw() {
    if (!this->beginWindow()) return;
```

```
// Existing code...

// TODO: Add Pause/Resume button
if (ImGui::Button("Pause")) {
    // ...
}
ImGui::SameLine();
if (ImGui::Button("Kill")) {
    // TODO: Show confirmation dialog
}

// TODO: Add priority dropdown
const char* priorities[] = { "Low", "Normal", "High", "Real-time" };
static int currentPriority = 1;
if (ImGui::Combo("Priority", &currentPriority, priorities, 4)) {
    // TODO: Update process priority
}

this->endWindow();
}
```

Discussion questions:

- How do real OSes implement process priority? (Hint: scheduling algorithms)
- What are the security implications of allowing users to kill any process?
- How would you implement a "Force Kill" feature for unresponsive processes?

Exercise 4: Task Manager Performance Graphs

Objective: Enhance TaskManagerUI with additional performance visualizations.

Requirements:

1. Add a "Disk Usage" graph
2. Add a "Network Usage" graph
3. Display min/max/average values for each metric
4. Allow user to change graph time range (e.g., 30s, 1m, 5m)
5. Add a "Pause" button to freeze the graphs

Starter code:

```
void TaskManagerUI::drawPerformanceTab() {
    // Existing CPU and memory graphs...

    // TODO: Add disk usage graph
    ImGui::Text("Disk Usage");
    ImGui::PlotLines("##Disk", diskHistory.data(), diskHistory.size(),
                    0, nullptr, 0.0f, 100.0f, ImVec2(0, 80));

    // TODO: Display statistics
    ImGui::Text("Min: %.1f%% | Max: %.1f%% | Avg: %.1f%%",
                getMin(diskHistory), getMax(diskHistory), getAvg(diskHistory));

    // TODO: Add time range selector
    const char* ranges[] = { "30s", "1m", "5m" };
    static int currentRange = 0;
    if (ImGui::Combo("Time Range", &currentRange, ranges, 3)) {
        resizeHistoryBuffers(currentRange);
    }
}
```

Discussion questions:

- How does Windows Task Manager implement its performance graphs?
- What are the trade-offs between graph resolution and memory usage?
- How would you implement a "Export to CSV" feature?

Exercise 5: Multi-Desktop Support

Objective: Implement virtual desktops (like Windows 10+ Task View).

Requirements:

1. Support at least 3 virtual desktops
2. Each desktop has its own set of open windows
3. Add a desktop switcher UI (e.g., thumbnails in taskbar)

- 4. Animate transitions between desktops (e.g., slide effect)
- 5. Allow dragging windows between desktops

Starter code:

```
class DesktopManager {
public:
    void addDesktop() {
        desktops.push_back(std::make_shared<Desktop>());
    }

    void switchToDesktop(int index) {
        if (index >= 0 && index < desktops.size()) {
            currentDesktopIndex = index;
            // TODO: Animate transition
        }
    }

    void renderCurrentDesktop() {
        if (currentDesktopIndex < desktops.size()) {
            desktops[currentDesktopIndex]->draw();
        }
    }

private:
    std::vector<std::shared_ptr<Desktop>> desktops;
    int currentDesktopIndex = 0;
};
```

Discussion questions:

- How do real OSes implement virtual desktops? (Hint: window visibility flags)
- What are the memory implications of multiple desktops?
- How would you implement a "Show all desktops" overview (like macOS Mission Control)?

Activity: Group Presentation

Objective: Each student group implements one of the exercises above and presents to the class.

Format:

1. **Demo** (5 minutes): Show your implementation running
2. **Code walkthrough** (5 minutes): Explain key design decisions
3. **Challenges** (3 minutes): Discuss difficulties encountered and how you solved them
4. **Q&A** (2 minutes): Answer questions from classmates and instructor

Evaluation criteria:

- **Functionality:** Does it meet all requirements?
- **Code quality:** Is the code well-structured and readable?
- **UI/UX:** Is the interface intuitive and responsive?
- **Presentation:** Is the explanation clear and engaging?

Activity: Performance Profiling

Objective: Measure and optimize the CSOPESY emulator's performance.

Tasks:

1. **Measure frame rate:** Use `ImGui::GetIO().Framerate` to display FPS
2. **Identify bottlenecks:** Use a profiler (e.g., Visual Studio Profiler, Valgrind) to find slow functions
3. **Optimize:** Reduce unnecessary computations in the render loop
4. **Compare:** Measure FPS before and after optimization

Guiding questions:

- What is the target frame rate for a desktop application? (Hint: 60 FPS)
- What operations should be done every frame vs. less frequently?
- How does the number of open windows affect performance?

Report format:

- **Baseline:** FPS with 0, 5, 10, 20 windows open
- **Bottlenecks:** Top 3 slowest functions (with profiler screenshots)
- **Optimizations:** What you changed and why
- **Results:** FPS improvement after optimization

12. Discussion Questions

Conceptual Questions

1. **Immediate vs Retained Mode:**
 - Why is immediate-mode GUI simpler to implement than retained-mode?
 - In what scenarios would retained-mode be preferable?
 - How does ImGui's paradigm relate to functional programming concepts?
2. **Polling vs Event-Driven:**
 - Why do real-time applications (games, simulations) use polling?
 - Why do event-driven applications (text editors, IDEs) wait for events?
 - Can you design a hybrid approach that combines both?
3. **OS Design:**
 - How does the CSOPESY emulator's window management compare to Windows' DWM?
 - What are the advantages of a compositing window manager?
 - How would you implement window transparency and blur effects?
4. **Performance:**
 - What is the performance cost of rebuilding the UI every frame?
 - How can you minimize draw calls in ImGui?
 - What is the role of GPU acceleration in modern UI rendering?

Implementation Questions

5. **DPI Scaling:**
 - Why is DPI scaling necessary for modern applications?
 - How does the CSOPESY emulator's UIConfig approach compare to OS-level scaling?
 - What are the challenges of supporting multiple monitors with different DPIs?
6. 9. **State Management:**
 - How does ImGui manage widget state (e.g., text input buffers)?
 - What is the role of widget IDs in ImGui?
 - How would you implement persistent window positions across sessions?
7. 10. **Rendering Pipeline:**
 - What is the purpose of double buffering (glfwSwapBuffers())?
 - How does VSync affect frame rate and input latency?
 - What is the difference between immediate-mode rendering and deferred rendering?